Introduction to Multi-Agent Communication (JADE)

Prof. Abderrahim Waga - UEMF

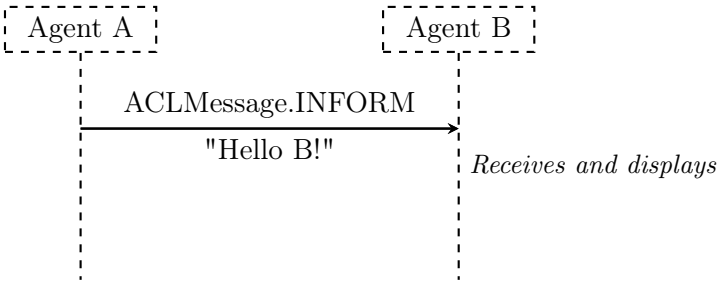
Academic Year 2025-2026

Lab Objectives

- Understand the ACL (Agent Communication Language) message exchange mechanism.
- Implement unidirectional communication (without reply).
- Implement bidirectional communication (with reply).
- Coordinate exchanges between three distinct agents.

Exercise 1: Unidirectional Communication (1 to 1)

Objective: Agent A (Sender) sends a simple information message to Agent B (Receiver). Agent B reads the message and displays it, without replying.

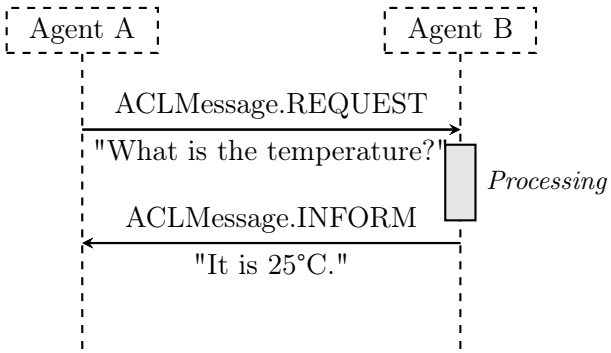


Tasks to do:

1. Create the `AgentB` class. In its `setup()` method, add a cyclic behavior (`CyclicBehaviour`) that uses the `receive()` method to listen for incoming messages.
2. Create the `AgentA` class. In its `setup()` method, add a one-shot behavior (`OneShotBehaviour`).
3. In this behavior, instantiate an `ACLMessage` object of type `INFORM`.
4. Set the receiver (AID of Agent B) and the message content ("Hello B!"), then send it using `send()`.
5. Launch JADE, start Agent B, then Agent A, and observe the console.

Exercise 2: Bidirectional Communication

Objective: Agent A requests information from Agent B. Agent B processes the request and sends back a reply.

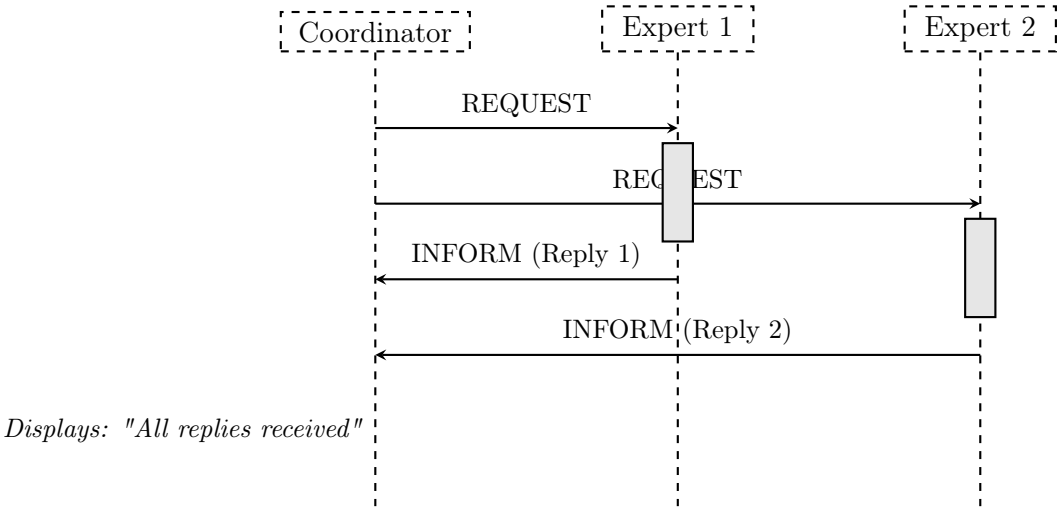


Tasks to do:

1. Modify `AgentA` so that its initial message is of type `REQUEST`. After sending, Agent A must switch to a listening state (`receive()`) to wait for the reply.
2. Modify `AgentB`: when it receives a `REQUEST`, it must use the `createReply()` method on the received message to generate an automatic reply to the correct sender.
3. Agent B sets the type of its reply to `INFORM`, adds the content ("It is 25°C."), and sends it.

Exercise 3: Coordination of 3 Agents (1 to 2)

Objective: A Coordinator Agent sends two different tasks to two expert agents and waits for both of their replies before concluding.



Tasks to do:

1. Create three agents: `CoordinatorAgent`, `Expert1`, and `Expert2`.
2. **Coordinator side:** In a `OneShotBehaviour`, send a `REQUEST` to Expert 1 and another `REQUEST` to Expert 2.

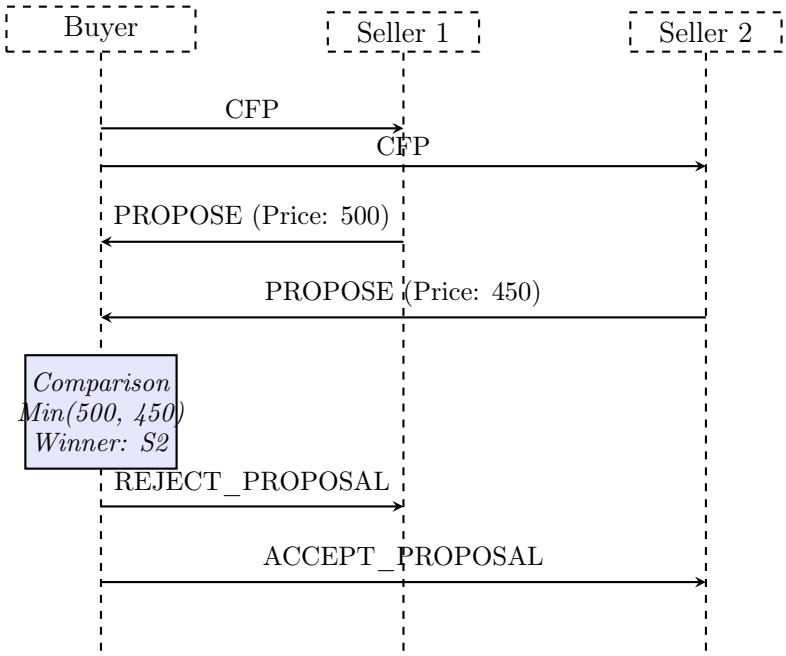
3. Then add a cyclic behavior to the coordinator to listen for replies. Use a **counter** initialized to 0. Each time it receives an `INFORM`, the counter increments.

4. If the counter reaches 2, the coordinator displays the global result and destroys its behavior.

5. **Experts side:** They work exactly like Agent B from exercise 2. They receive a message, use `createReply()` and send back an `INFORM`.

Exercise 4: Decision Making and Negotiation

Introduce decision making. A Buyer Agent (Coordinator) wants to buy parts at the best price. It issues a Call For Proposal (*CFP*) to two Seller Agents (Experts). It compares the proposed prices, accepts the cheapest one, and rejects the other.



Tasks to do:

1. **Buyer side (Initiator):**

   - Create a message of type `ACLMessage.CFP` and send it to Seller 1 and Seller 2.
   - Switch to listening mode. Retrieve messages of type `PROPOSE`.
   - Extract the message content (the price) with `msg.getContent()` and convert it to an integer (`Integer.parseInt()`).
   - Compare the two prices. Keep in memory the identifier (`AID`) of the agent that proposed the lowest price.
   - Use `createReply()` on the received messages to send an `ACCEPT_PROPOSAL` to the winner and a `REJECT_PROPOSAL` to the loser.

2. **Sellers side (Participants):**

   - They listen for messages of type `CFP`.
   - Upon receiving a `CFP`, they generate a price (you can hardcode a different fixed price for S1 and S2, or use `Math.random()`).
   - They reply with a `PROPOSE` message containing this price in text format.

- They go back to listening to wait for acceptance (`ACCEPT_PROPOSAL`) or rejection (`REJECT_PROPOSAL`) and display the result in the console (e.g., *"Seller 2: Hooray, I won the deal!"*).

JADE Tip: To filter only a certain type of message upon reception, you can use **MessageTemplate** (e.g., `MessageTemplate.MatchPerformative(ACLMessage.PROPOSE)`).